Advanced Communication, Performance and Security

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Practical Lab: Mastering the Restricted Network

Lab Context: Imagine you are a student at the University. You are connected to **eduroam**, a network that is secure but very restrictive. You want to run a project where a sensor at your home syncs data to a server, and you need to access a private database from your laptop while sitting in the University library. Direct connections are blocked. You must use your skills to **measure**, **sync**, and **tunnel** through these limitations.

Part 0: Setup & Infrastructure

Learning Objective: Deploy a simulated "Restricted Enterprise Network" using containers.

We will use Docker to create our mini-university ecosystem.

1. Create the Lab Network

```
$ docker network create --subnet=172.25.0.0/16 lab-net
```

2. Launch the Infrastructure

Create a folder `class08-lab` and a `compose.yml` file:

```yaml
services:
  # Represents your Home Server (the destination for backups)
  home-server:
    build:
      context: ./home-server
    container_name: home-server
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Lisbon
      - USER_NAME=student
      - PASSWORD_ACCESS=true
      - USER_PASSWORD=studentpass
    ports:
      - "2222:2222"

    networks:
      lab-net:
        ipv4_address: 172.25.0.10

  # Represents your Laptop (connected to restricted eduroam)
  laptop:
    image: alpine:latest
    container_name: laptop
```

```yaml
    networks:
      lab-net:
        ipv4_address: 172.25.0.20
    command: sh -c "apk add rsync openssh-client iperf3 mtr tcpdump bash curl sshpass && s

  # Represents a remote traffic target
  internet-target:
    image: alpine:latest
    container_name: internet-target
    networks:
      lab-net:
        ipv4_address: 172.25.0.30
    command: sh -c "apk add iperf3 && iperf3 -s"

  # Represents a Private Database (isolated from everyone)
  private-db:
    image: alpine:latest
    container_name: private-db
    networks:
      lab-net:
        ipv4_address: 172.25.0.40
    command: sh -c "apk add python3 && echo 'SENSITIVE_STUDENT_DATA' > secret.txt && pytho

  # Represents a Load Balancer for University Services
  uni-gateway:
    image: nginx:alpine
    container_name: uni-gateway
    ports:
      - "8081:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      lab-net:
        ipv4_address: 172.25.0.100


networks:
  lab-net:
    external: true
```

3. Create the Configuration Files Create a subfolder `home-server` and a `Dockerfile` inside it:

```dockerfile
FROM lscr.io/linuxserver/openssh-server:latest
RUN apk add --no-cache rsync
```

Create an `nginx.conf` file in the main `class08-lab` folder:

```nginx
events {
    worker_connections 1024;
}

http {
    upstream university_services {
        server 172.25.0.40:5432;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://university_services;
            proxy_set_header Host $host;
```

```
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

4. Start the Environment

```
$ docker compose up -d
```

Part 1: Performance & Monitoring

Goal: Prove why a network feels "slow" using data.

Exercise 1: Throughput Testing (`iperf3`) **Real-world Scenario:** You are trying to download a large dataset in the library, but it's taking forever. Is it the server or the Wi-Fi?

1. Run the client from your `laptop` to the `internet-target`:

   ```
   $ docker exec -it laptop iperf3 -c 172.25.0.30
   ```

2. **Analysis:** If the throughput is 1Gbps, the network is perfect. If it's 1Mbps, you found the bottleneck.

Exercise 2: Real-time Analysis (`mtr`) **Real-world Scenario:** Your video call keeps dropping. You suspect a faulty router in the University building.

1. Run `mtr` from your `laptop` to the `home-server`:

   ```
   $ docker exec -it laptop mtr 172.25.0.10
   ```

2. Look at the loss column. If loss starts at the first hop, the local Access Point is bad.

Part 2: Efficient Synchronization (`rsync`)

Goal: Move data without wasting time or bandwidth.

Exercise 3: Smart Backups **Real-world Scenario:** You have a 100MB project file. You only changed one paragraph. On eduroam, upload bandwidth is limited.

1. Create a large file on your laptop: `docker exec -it laptop dd if=/dev/urandom of=/project.pdf bs=1M count=10`.
2. Sync to your `home-server` (pass `studentpass`):

   ```
   $ docker exec -it laptop rsync \
   -avz -e 'ssh -p 2222' /project.pdf student@172.25.0.10:/config/
   ```

3. Modify the file slightly and sync again. Notice how much faster it is.

Part 3: Bypassing Limitations (SSH)

Goal: Reach what is hidden or blocked.

Exercise 4: Local Port Forwarding **Real-world Scenario:** You need to query the `private-db` for your thesis, but eduroam blocks port 5432. You have SSH access to your `home-server`.

1. Open the tunnel from your **host**: `ssh -p 2222 -L 9000:172.25.0.40:5432 student@localhost`.
2. Access `http://localhost:9000` on your machine. You have successfully "bridged" into the isolated DB.

Exercise 5: Dynamic SOCKS Proxy **Real-world Scenario:** The University blocks a specific research site you need. You use your home connection to browse through it.

1. Create the proxy: `ssh -p 2222 -D 1080 student@localhost`.
2. Configure your laptop's `curl` to use it:

   ```
   $ curl --proxy socks5h://localhost:1080 http://172.25.0.40:5432
   ```

Part 4: Advanced Reliability

Exercise 6: Load Balancing **Real-world Scenario:** Thousands of students access the grades portal at the same time.

1. Use the `uni-gateway` to distribute traffic and test what happens if one server goes offline.